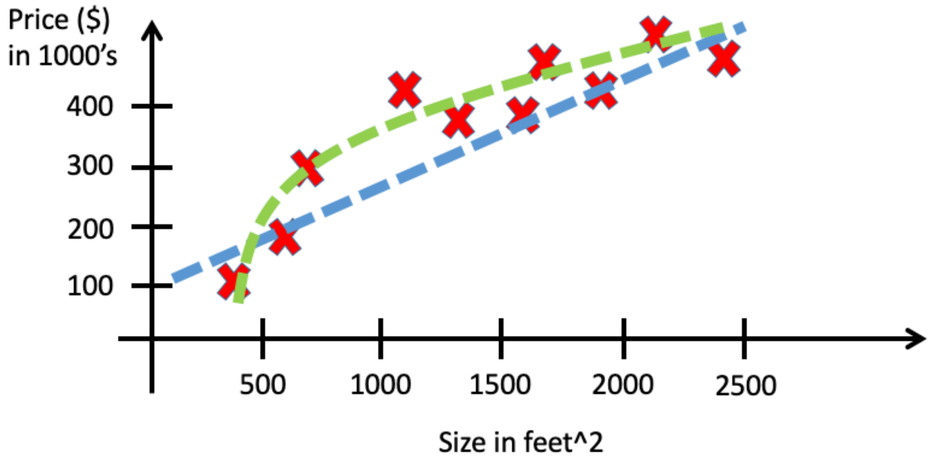
Linear Regression

Hoda Eldardiry

Machine Learning

What problem are we solving?

- **Goal:** Predict a real-valued output from data
- Given input features, predict a continuous target
- Examples:
  - latency vs input size
  - price vs features
  - score vs study hours



- *Regression: learning a function from data for prediction*

Where does regression fit in ML?

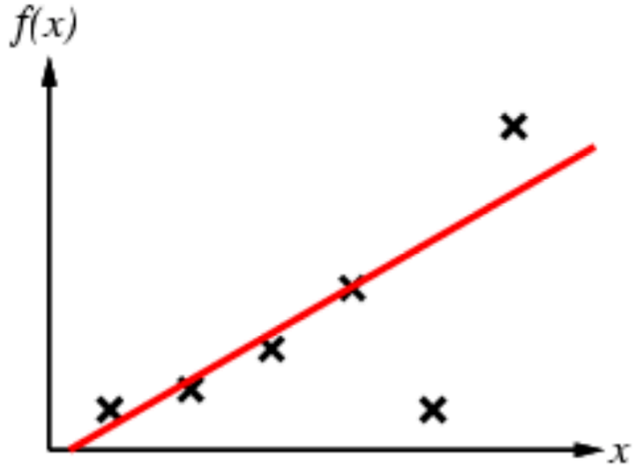
- Supervised learning
- Training data (x_i, y_i)
- Learn parameters from labeled examples
- *Today: linear regression as the simplest learning algorithm*

Learning = parameter estimation

- Choose a hypothesis class
- Define a loss (cost) function
- Use optimization to find best parameters
- *This pattern repeats across ML models*

Hypothesis class: straight lines

- We restrict ourselves to linear functions
- $\hat{y} = h_\theta(x) = \theta_0 + \theta_1 x$
  - θ_0: bias (intercept)
  - θ_1: weight (slope)
- *Assumption: relationship is approximately linear*

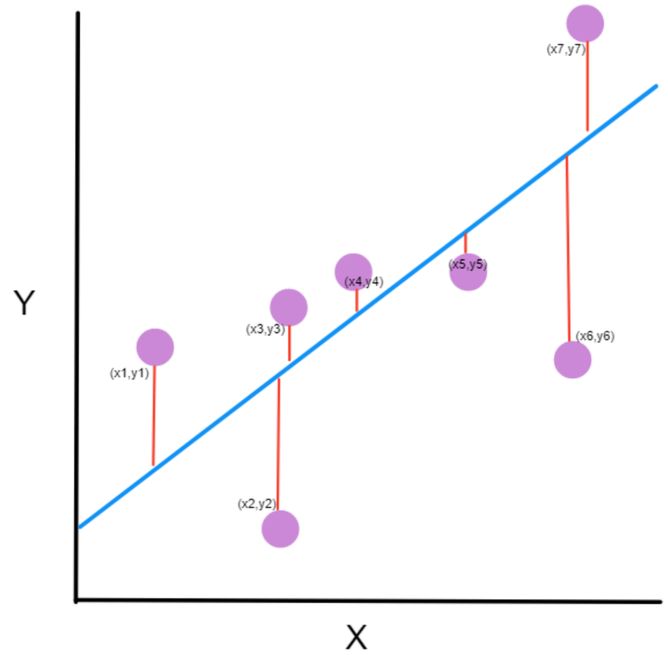


Different parameters = different models

- Each choice of θ gives a different line
- Learning = choosing the *best* line for the data
- *Question: what does “best” mean?*

Measuring error: residuals

- Prediction error for example *i*: $\hat{y}_i - y_i$
- Geometrically: vertical distance from point to line
- *We want small errors on average*



Cost function: Mean Squared Error (MSE)

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (h_\theta(x_i) - y_i)^2$$

- Squares penalize large errors
- Smooth and differentiable
- *Learning = minimize this cost*

Learning as optimization

- Parameters θ are variables
- Cost function $J(\theta)$ is the objective
- *This is an optimization problem*

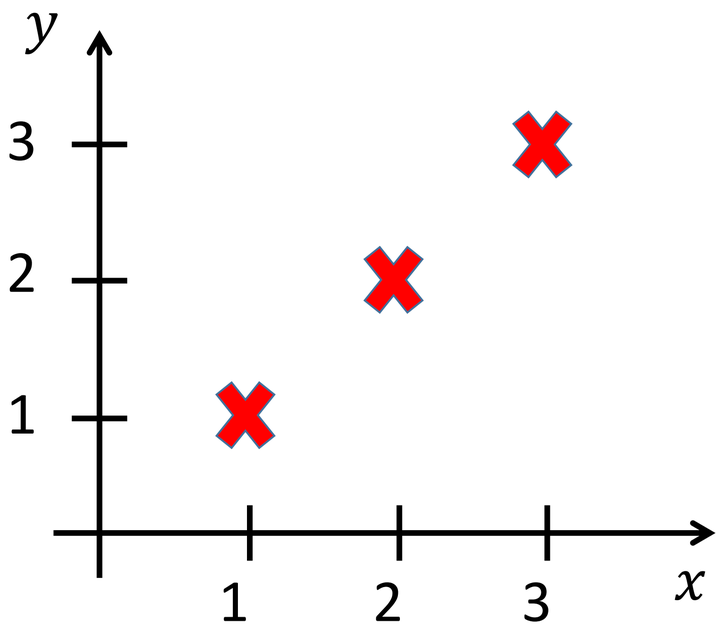
$$\arg\min_{\theta} J(\theta)$$

Visualizing the cost surface

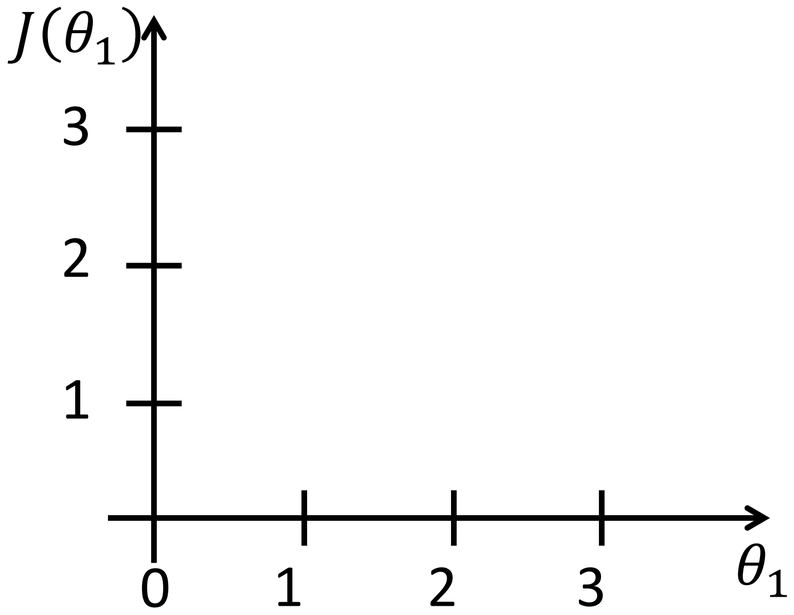
- One parameter —> curve
- Two parameters —> bowl-shaped surface
- Global minimum exists for linear regression
- *Nice property: convex optimization*

Single parameter example

$h_\theta(x)$, function of x



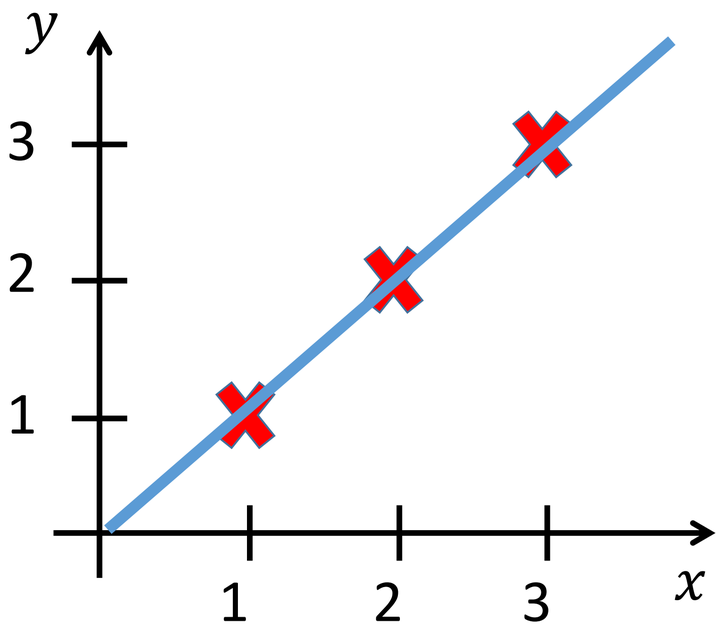
$J(\theta_1)$, function of θ_1



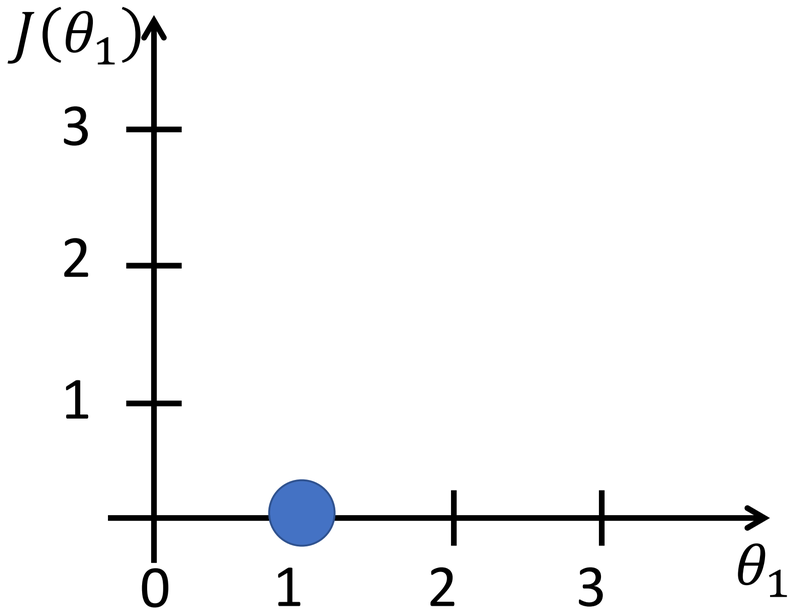
Slide credit: Andrew Ng

Single parameter example

$h_\theta(x)$, function of x



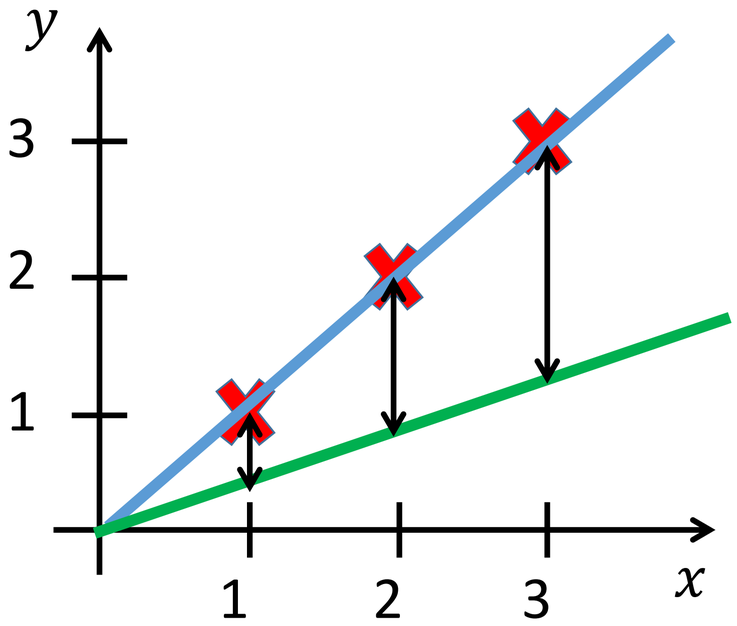
$J(\theta_1)$, function of θ_1



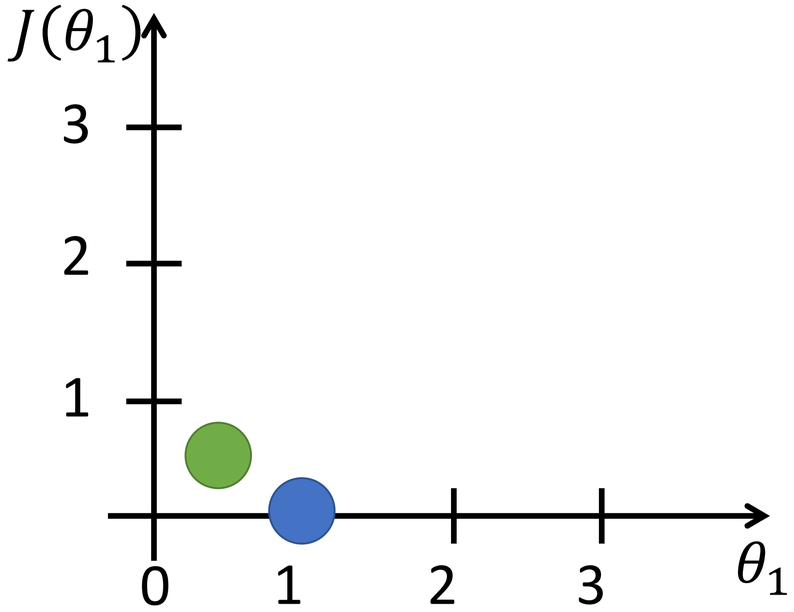
Slide credit: Andrew Ng

Single parameter example

$h_\theta(x)$, function of x



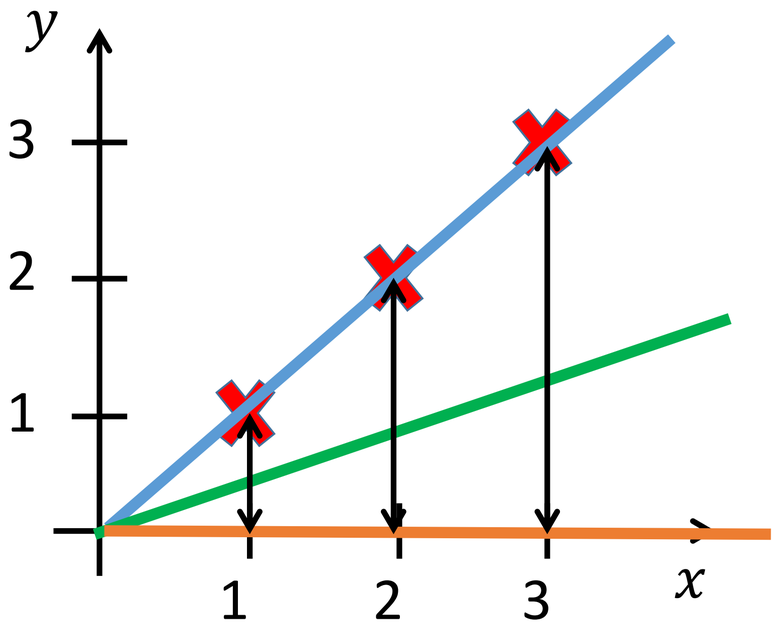
$J(\theta_1)$, function of θ_1



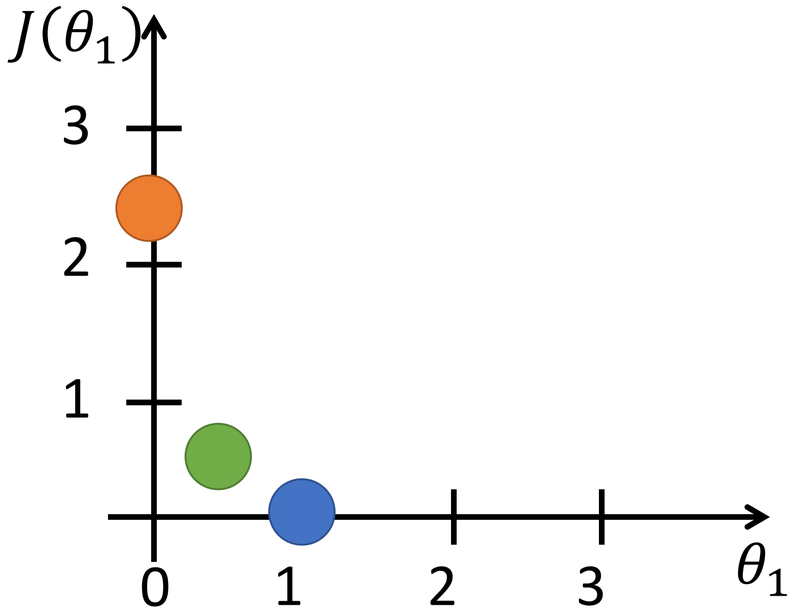
Slide credit: Andrew Ng

Single parameter example

$h_\theta(x)$, function of x



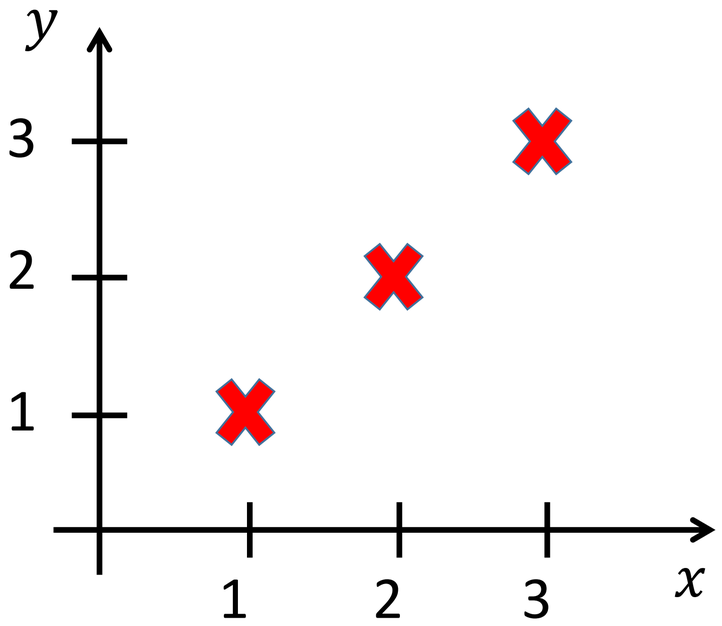
$J(\theta_1)$, function of θ_1



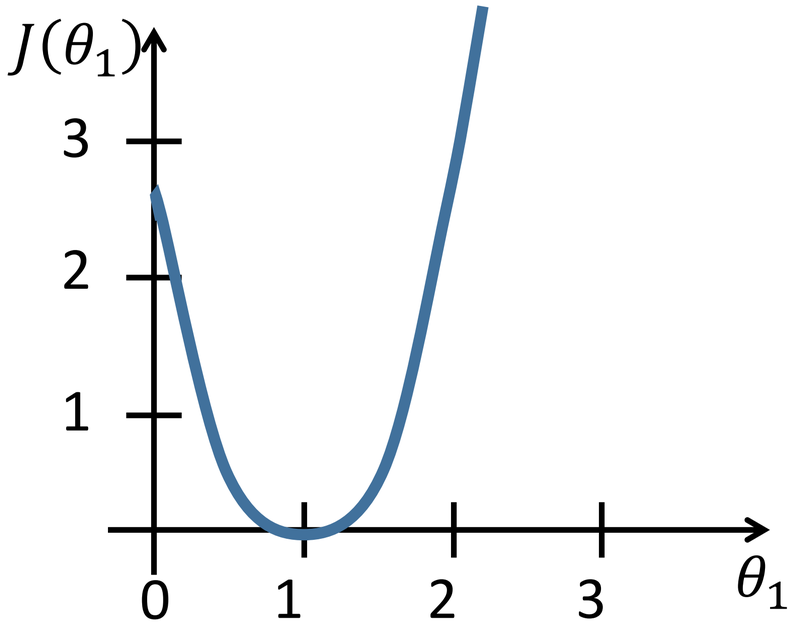
Slide credit: Andrew Ng

Single parameter example

$h_\theta(x)$, function of x



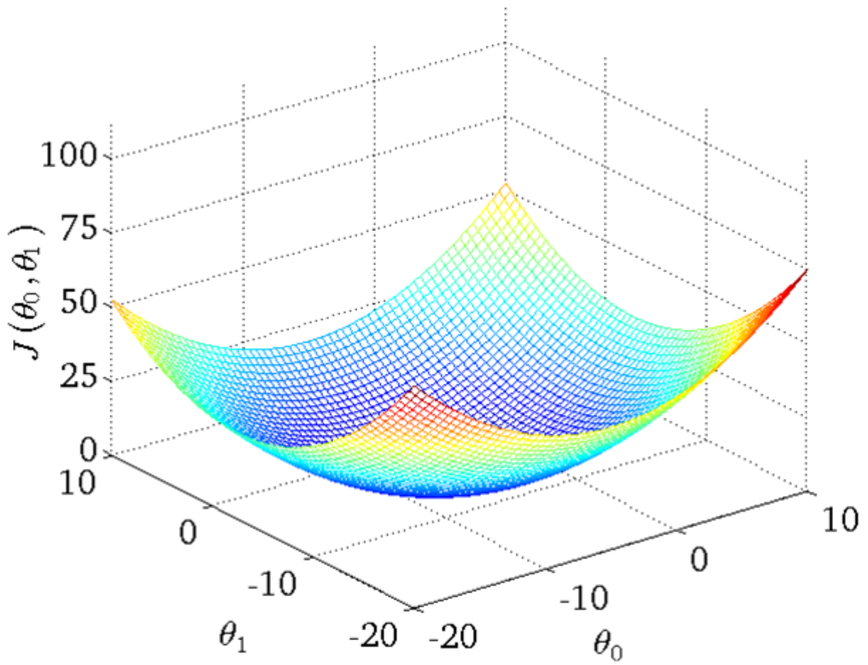
$J(\theta_1)$, function of θ_1



Slide credit: Andrew Ng

Two parameters

Cost function

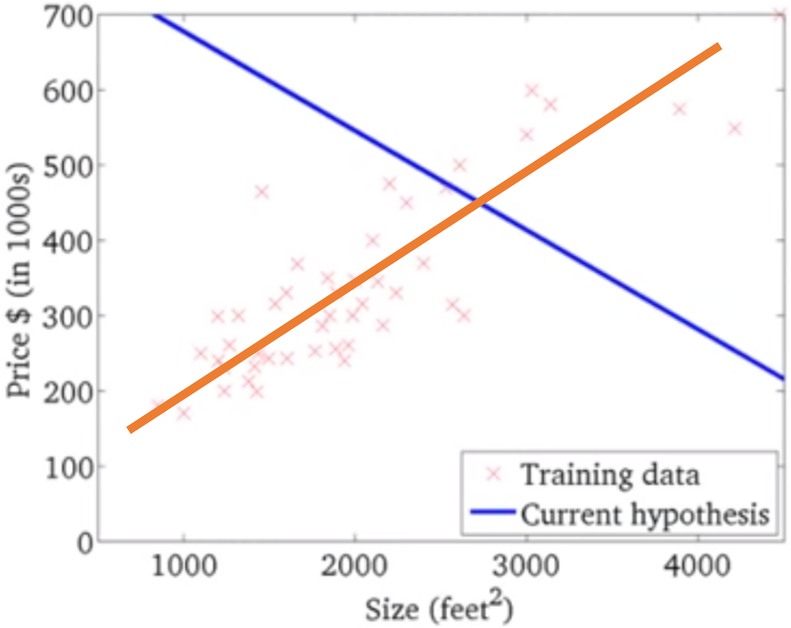


Slide credit: Andrew Ng

Two parameters

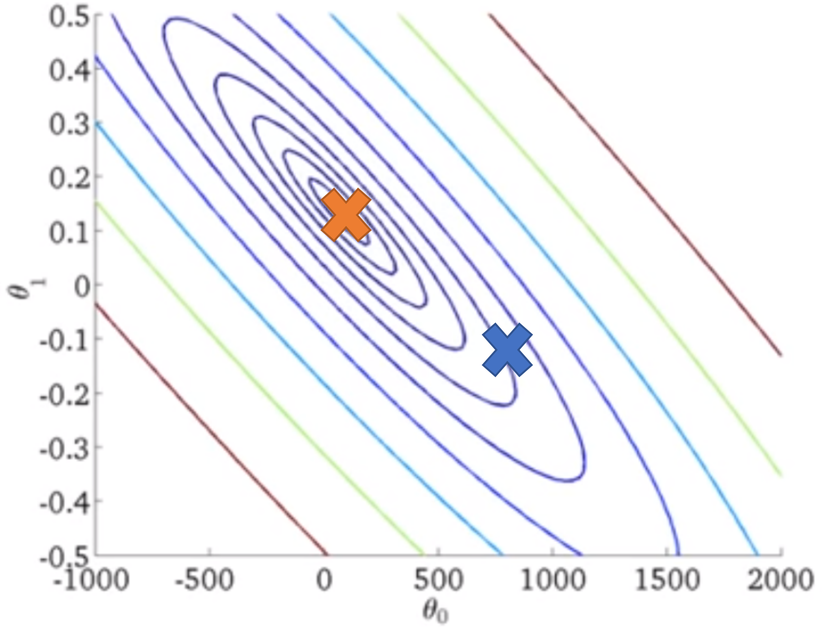
$h_\theta(x)$

(for fixed θ_0, θ_1, this is a function of x)



$J(\theta_0, \theta_1)$

(function of the parameters θ_0, θ_1)



How do we find good θ_0, θ_1 that minimize $J(\theta_0, \theta_1)$?

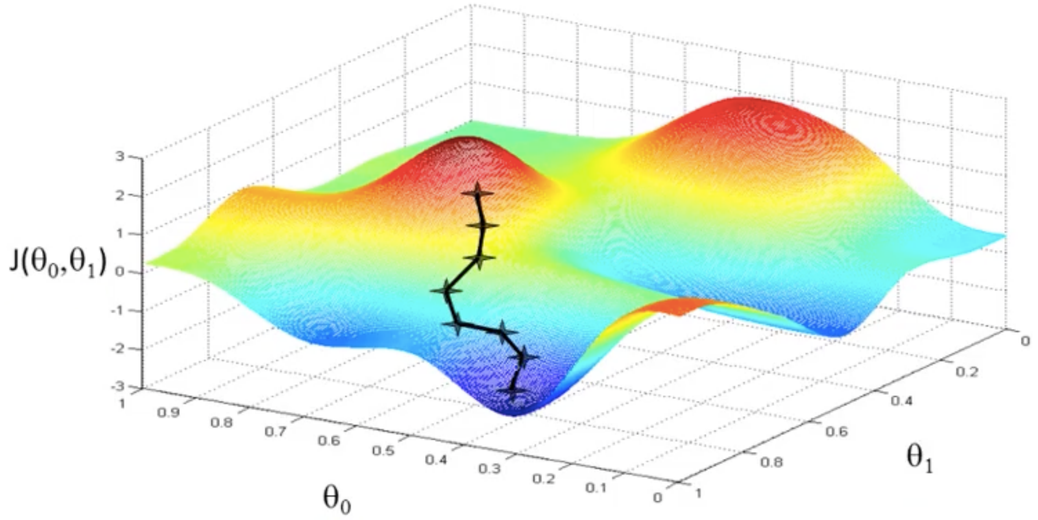
Slide credit: Andrew Ng

How do we find the minimum?

- Closed form exists, but we focus on:
- *Gradient Descent: general purpose optimization algorithm used in ML*

Gradient Descent (intuition)

- Start with random parameters
- Repeatedly move in direction of steepest descent
- Step size can be controlled by learning rate α
- *Same idea used in neural networks*



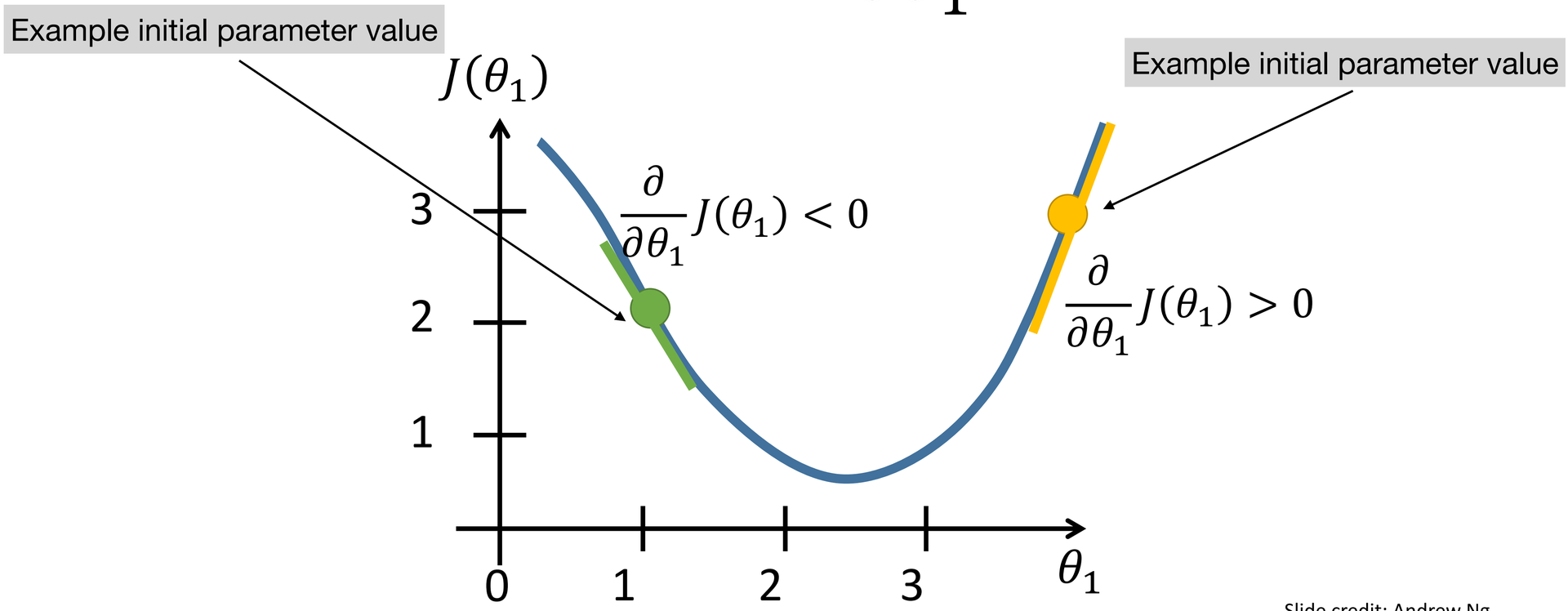
Slide credit: Andrew Ng

Gradient Descent update rule

- Repeat until convergence: $\theta_j := \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$
- α: learning rate
- Derivative = slope of cost surface

Simplified example: parameter update

$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$



Slide credit: Andrew Ng

Gradient Descent for linear regression

- For each iteration:

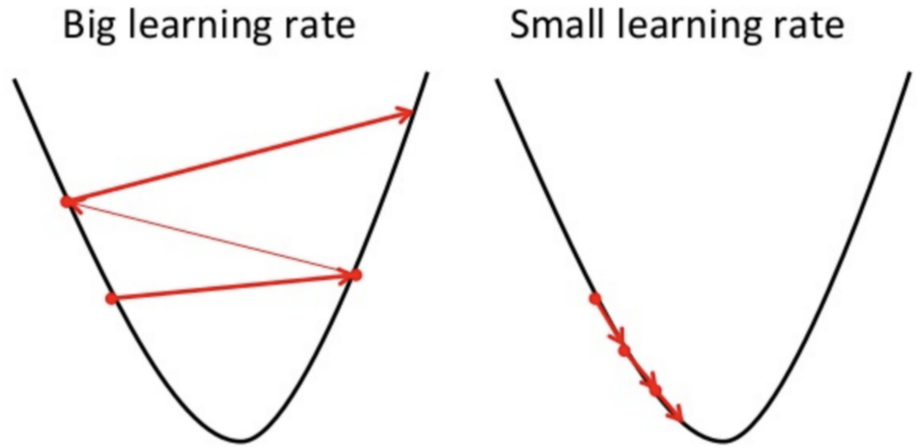
$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum (h_\theta(x_i) - y_i)$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum (h_\theta(x_i) - y_i) x_i$$

- *Update parameters simultaneously*

Learning rate matters

- Too small —> slow convergence
- Too large —> divergence
- *Trade-off: speed vs stability*



Parameter selection update as gradient descent progresses with varying learning rates

Batch Gradient Descent

- Each update uses all training examples
- Deterministic, stable
- Can be very expensive for large datasets
- *Variants: stochastic, mini-batch*

Multiple features

- Generalize to: $\hat{y} = \theta_0 + \theta_1 x_1 + \cdots + \theta_n x_n$
- Same learning framework
- More parameters
- Same ideas

What assumptions are we making?

- Linear relationship
- Features capture relevant signal
- Noise is unstructured
- *Assumptions simplify learning, but can fail*

When linear regression breaks

- Nonlinear patterns
- Outliers with high leverage
- Extrapolation far from training data
- *Model mismatch = systemic error*

Regression is not causation

- Regression captures association
- Not proof of cause-effect
- Especially important in ML-driven decisions

Big picture takeaway

- Linear regression is a learning algorithm
- Defined by:
  - Hypothesis class
  - Loss function
  - Optimization method
- *This template scales to most ML models*

What’s next?

- Classification
- Logistic regression
- *Same ideas, new loss functions*